BitTorrent Protocol: From UDP Tracker Discovery to TCP Peer Wire Data Exchange

Connection Initialization Request and Response Memory Layout

The UDP tracking process begins with a Connection Request. To prevent IP spoofing and mitigate distributed denial-of-service amplification attacks, a tracker will not accept an announce request until the client has successfully proven its source IP address by obtaining a time-limited connection ID.

The client constructs a precisely 16-byte UDP datagram and dispatches it to the tracker's IP and port.

Offset	Size (Bytes)	Data Type	Field Name	Description and Value
0	8	uint64_t	protocol_id	A fixed magic constant identifying the protocol: 0x41727101980.
8	4	uint32_t	action	The operation to perform; set to 0 to denote a connect request.
12	4	uint32_t	transaction_id	A randomized 32-bit integer generated by the client.

Upon receiving this datagram, the tracker validates the 64-bit protocol_id. If the magic constant matches, the tracker generates a randomized 64-bit connection_id bound to the client's source IP and responds with a 16-byte Connect Response.

Offset	Size (Bytes)	Data Type	Field Name	Description and Value
0	4	uint32_t	action	Mirrors the request; set to 0 for connect.
4	4	uint32_t	transaction_id	Must perfectly match the client's generated transaction ID.
8	8	uint64_t	connection_id	The secure session token, typically valid for 60 seconds.

Because UDP datagrams may be lost in transit, clients must implement a robust retry mechanism. The specification dictates an exponential backoff algorithm, where the timeout begins at 15 seconds and doubles for each subsequent retry (15 * 2^n), up to a maximum of 8 rounds, before the tracker is marked as unresponsive.

The Announce Request and Response Architecture

Armed with a valid connection_id, the client proceeds to generate an Announce Request. This 98-byte payload encapsulates all necessary metrics regarding the client's current status within the swarm, while simultaneously requesting a list of active peers.

Offset	Size (Bytes)	Data Type	Field Name	Description and Value

0	8	uint64_t	connection_id	The 64-bit token successfully retrieved from the Connect phase.
8	4	uint32_t	action	Set to 1 to denote an Announce Request.
12	4	uint32_t	transaction_id	A newly generated, randomized 32-bit integer.
16	20	uint8_t	info_hash	The 20-byte SHA-1 hash of the bencoded info dictionary.
36	20	uint8_t	peer_id	The unique 20-byte string identifying the local client instance.
56	8	uint64_t	downloaded	The cumulative total of successfully verified bytes downloaded.
64	8	uint64_t	left	The exact number

				of bytes required to reach 100% completion.
72	8	uint64_t	uploaded	The cumulative total of bytes uploaded to other peers.
80	4	uint32_t	event	The current state: 0 (none), 1 (completed), 2 (started), 3 (stopped).
84	4	uint32_t	ip_address	Often set to 0 to allow the tracker to derive the IP from the UDP header.
88	4	uint32_t	key	A randomized key used for identifying the client across IP changes.
92	4	int32_t	num_want	The requested peer count. A value of -1 instructs the tracker to use its default.

96	2	uint16_t	port	The TCP port the client has bound to listen for incoming connections.

The proper sizing of the statistical metrics (downloaded, left, uploaded) as 64-bit integers is vital. Modern torrents frequently exceed 4.29 GB, meaning 32-bit variables would immediately overflow, transmitting negative values to the tracker and destroying swarm statistics. If the client has not yet parsed the metadata and does not know the file size, left can be highly problematic. In some architectures, passing -1 to represent an unknown size causes deserialization failures if the tracker is strictly expecting an unsigned 64-bit boundary; resolving this requires explicitly handling null or zero values in early announce phases.

Assuming the Announce Request is valid, the tracker responds with an Announce Response. This message consists of a fixed 20-byte header, followed by a dynamic payload containing the requested peers.

Offset	**Size (Bytes)**	**Data Type**	**Field Name**	**Description and Value**
0	4	uint32_t	action	Mirrors the request; set to 1 for announce.
4	4	uint32_t	transaction_id	Must perfectly match the client's generated transaction ID.
8	4	uint32_t	interval	The mandatory minimum wait time, in seconds, before the next

				announce.
12	4	uint32_t	leechers	The total number of peers who have not yet completed the download.
16	4	uint32_t	seeders	The total number of peers holding 100% of the pieces.
20	Variable	uint8_t	peers	The compact representation of peer IP addresses and connection ports.

The peers payload demands careful byte manipulation. To minimize packet sizes and prevent UDP fragmentation, trackers return peers in a highly compressed format. For standard IPv4 endpoints, each peer is represented by a strict 6-byte stride: the first 4 bytes correspond to the IPv4 address, and the remaining 2 bytes represent the TCP port. A response yielding 50 peers will thus have a 300-byte payload appended to the 20-byte header.

However, with the exhaustion of the IPv4 address space, BEP 15 was updated to accommodate IPv6 formats. When operating over an IPv6 stack, the structure of the header remains identical, but the peer stride expands to 18 bytes. The first 16 bytes encode the IPv6 address, while the final 2 bytes encode the port. The client's socket implementation must dynamically determine the parsing stride by evaluating the address family (AF_INET vs. AF_INET6) of the received UDP datagram.

The Data Plane: TCP Connection Establishment and Handshake Memory Layout

Having extracted a list of IPs and ports from the UDP tracker, the client transitions to the data plane. The client initiates asynchronous TCP connections to the targeted peers, creating a

mesh network topology.

The BitTorrent Peer Wire Protocol demands that the absolute first bytes transmitted across any new TCP connection must be the Handshake. The protocol operates as a strict state machine; if a peer transmits any data before a valid handshake, the socket must be immediately destroyed. The Handshake is fixed at exactly 68 bytes for version 1.0 of the protocol.

Offset	**Size**	**Data Type**	**Field Name**	**Memory Content and Architectural Purpose**
0	1	uint8_t	pstrlen	0x13 (Decimal 19). The length of the immediately following protocol string.
1	19	uint8_t	pstr	The literal ASCII byte sequence "BitTorrent protocol".
20	8	uint8_t	reserved	64 bits representing a dense boolean array used to negotiate protocol extensions.
28	20	uint8_t	info_hash	The SHA-1 hash retrieved from the metainfo file, uniquely identifying the swarm.

48	20	uint8_t	peer_id	The 20-byte alphanumeric identifier uniquely representing the initiating client.

The receiving peer evaluates this stream linearly. It first validates pstrlen and pstr to ensure the incoming connection is a legitimate BitTorrent client rather than a port scanner or web crawler. Next, it isolates the info_hash starting at offset 28. Because a single client application may be seeding hundreds of different torrents simultaneously across the same TCP listening port, the info_hash is critical for internal routing. If the client determines it is not actively serving the requested info_hash, it drops the connection. Finally, the peer_id at offset 48 allows the receiver to verify the client's identity. peer_id strings often follow conventions, such as Azureus-style encoding (-AZ2060- followed by 12 random digits), which passively identifies the client software and version.

Protocol Extensions and Reserved Bitfield Negotiation

At offset 20 of the Handshake lies the reserved byte array. Originally, Bram Cohen's specification mandated that all 64 bits in this region must be identically set to zero (0x00). As the protocol evolved, these bytes were repurposed into a highly efficient negotiation matrix. Each bit maps to a specific BitTorrent Enhancement Proposal, allowing peers to dynamically detect mutually supported features.

Because the negotiation occurs sequentially, a client performs bitwise AND operations against these 8 bytes.

1. **Distributed Hash Table (DHT) Support (BEP 5):** If a client implements the trackerless DHT routing protocol, it sets the last bit of the 8th reserved byte. In a C-style memory array, this is expressed as reserved |= 0x01. If both peers flag this bit, they are permitted to dispatch PORT messages (Message ID 9), informing each other of the UDP port bound for DHT traffic.
2. **Fast Extension Suite (BEP 6):** The Fast Extension addresses inherent inefficiencies in the original Choking algorithm. It is negotiated by setting the third least significant bit of the last reserved byte, or reserved |= 0x04. This extension activates advanced piece synchronization techniques that bypass strict choking.
3. **Extension Protocol (BEP 10):** To prevent the core 1-byte message ID space from being exhausted, BEP 10 introduced a multiplexed extension framework. It is flagged by setting the 20th bit from the right. In zero-indexed array terms, this is evaluated as reserved & 0x10.

The Extension Protocol (BEP 10) and Extended Handshakes

If both peers evaluate the reserved & 0x10 bit as true, the connection unlocks Message ID 20, officially designated as the Extended Message. This mechanism allows infinite expandability without polluting the legacy message format.

A BEP 10 message payload begins with a single-byte extended_message_id. By definition, extended message ID 0 is universally reserved for the Extended Handshake. Thus, immediately following the bitfield exchange, peers transmit an Extended Handshake to configure dynamic routing.

The payload of the Extended Handshake is a bencoded dictionary containing operational metadata and client capabilities. The structure is entirely flexible, meaning parsers must ignore any unrecognized dictionary keys. A thoroughly populated Extended Handshake dictionary includes:

Dictionary Key	**Bencoded Type**	**Operational Description**
m	Dictionary	The core mapping matrix. It maps human-readable extension strings (e.g., ut_metadata, ut_pex) to dynamically assigned 1-byte integers.
p	Integer	Specifies the TCP port the remote client is listening on. This allows reverse connections if the current socket drops.
v	String	A human-readable identifier detailing the client software name and exact version

		number.
yourip	String	A raw binary representation of the requesting peer's public IP address (4 bytes for IPv4, 16 bytes for IPv6) as witnessed by the receiver.
reqq	Integer	Defines the absolute maximum number of unfulfilled block requests the peer can safely queue in memory before dropping packets.
metadata_size	Integer	The total byte length of the torrent's bencoded info dictionary. This is strictly required for resolving Magnet URIs, which lack a physical .torrent file.

The m dictionary is the linchpin of BEP 10. If Peer A maps ut_metadata to integer 3, and Peer B maps ut_metadata to integer 7, they can effortlessly exchange metadata. When Peer A sends metadata to Peer B, it encapsulates it under Extended Message ID 7. This dynamic integer binding cleanly bypasses protocol versioning conflicts.

Peer Wire Protocol: State Management and Message Framing

Following the initial handshakes, the TCP stream transitions into a perpetual state-machine loop. Because TCP is a continuous byte stream lacking inherent message boundaries, the BitTorrent protocol enforces a strict Length-Value-Payload (LVP) framing structure.

Every message transmitted over the wire—without exception—begins with a 4-byte big-endian integer denoting the length_prefix. This value dictates the exact number of bytes following the prefix that constitute the full message. Immediately following the length prefix is a 1-byte

message_id, and subsequently, the message-specific payload.

To parse this stream efficiently without blocking the thread, clients must read exactly 4 bytes from the socket, convert the data using ntohl (network-to-host long), allocate a buffer of that specific size, and continue reading from the socket until the buffer is saturated.

A unique exception is the Keep-Alive message. If the parser resolves the length_prefix to exactly 00000000, the message contains no message_id and no payload. Keep-Alives are injected into the TCP stream periodically—typically every two minutes—to prevent stringent NAT gateways from severing connections while the client is idle or performing intensive disk I/O operations.

State Matrix and the Choking Algorithm

The BitTorrent protocol relies on a cooperative algorithm to maximize global bandwidth and discourage "leeching" (downloading without uploading). This logic is governed by four state variables maintained independently per connection: am_choking (we refuse to send data), am_interested (we want data), peer_choking (they refuse to send data), and peer_interested (they want data). All connections initialize in a state of choked (1) and not interested (0).

Four zero-payload messages manipulate these states. For these messages, the length_prefix is always 00000001, indicating that only the 1-byte message_id follows.

- **Choke (ID 0):** <length=00000001><id=0>. The sender is severing the data pipeline. All pending block requests currently queued in memory are immediately considered discarded and must be re-requested elsewhere.
- **Unchoke (ID 1):** <length=00000001><id=1>. The sender has authorized data transfer and will fulfill incoming requests.
- **Interested (ID 2):** <length=00000001><id=2>. The sender desires pieces held by the receiver.
- **Not Interested (ID 3):** <length=00000001><id=3>. The sender requires none of the receiver's pieces.

The decision to emit Choke and Unchoke messages is controlled by the Tit-for-Tat Choking Algorithm. Every 10 seconds, the client calculates the rolling average of incoming download speeds from all connected peers. The client then unchokes the top four peers providing the fastest inbound data, rewarding them with outbound data. This 10-second interval is meticulously chosen to prevent "fibrillation"—the rapid toggling of TCP congestion windows that would destroy throughput. To discover new peers with potentially better bandwidth, the algorithm performs an "Optimistic Unchoke" every 30 seconds, randomly selecting a choked peer and unchoking them regardless of their upload history.

State Synchronization: Bitfield, Have, and Fast

Extensions

To make informed decisions about interest states, peers must maintain an exact geographical map of the torrent's data distribution.

Bitfield Calculation and Padding Logic

Immediately following the handshake, peers typically exchange a Bitfield message (ID 5). This is a highly compact memory array where every individual bit corresponds chronologically to a piece of the torrent. A bit set to 1 confirms possession and verification of the piece; a 0 indicates absence.

The byte-packing of the Bitfield requires precise logic. The first byte in the payload corresponds to pieces 0 through 7, mapping from the most significant bit to the least significant bit. The overall payload length is calculated using the ceiling function: ceil(total_pieces / 8).

If the torrent consists of 1,313 pieces, the payload will be 165 bytes (165 * 8 = 1,320 bits). Because the bitfield array spans 1,320 bits but only 1,313 pieces exist, the final 7 bits are deemed "spare". The protocol strictly mandates that these trailing spare bits must be padded with zeros (0). If a client's parser detects a 1 in the padding region, it signifies a corrupted client or a malicious actor, and the connection should be terminated. The complete 165-byte payload is prefixed with <length=000000A6><id=5> (where A6 represents decimal 166).

Incremental Synchronization

While the Bitfield provides bulk synchronization, the Have message (ID 4) handles incremental updates. The moment a client downloads the final block of a piece, reconstructs it, and verifies its SHA-1 hash, it blasts a Have message to all connected peers.

The memory layout of the Have message is fixed at 9 bytes:

- length_prefix: 00000005.
- message_id: 4.
- payload: A 4-byte big-endian integer representing the zero-based index of the completed piece.

The constant influx of Have messages enables the client to maintain a dynamically updating frequency map of the swarm. Using this map, the client executes the "Rarest First" algorithm, isolating pieces that are mathematically scarce across the peer network and prioritizing their download.

Fast Extension Enhancements (BEP 6)

The Fast Extension, negotiated during the TCP Handshake, introduces highly efficient alternatives to the Bitfield. If a client connects to the swarm with zero pieces (a pure leecher),

transmitting a bitfield comprised entirely of zeros wastes bandwidth. Conversely, a seeder transmitting a massive bitfield of ones incurs unnecessary overhead.

BEP 6 introduces Have None (ID 0x0F) and Have All (ID 0x0E). These 1-byte, zero-payload messages (<length=00000001><id=14> and <length=00000001><id=15>) entirely replace the Bitfield message, eliminating bulk overhead upon connection establishment.

Furthermore, BEP 6 restructures Choking semantics. Traditionally, when a Choke message is received, the client must discard all pending requests. This creates race conditions where valuable data is silently dropped. BEP 6 mandates the use of the Reject Request message (ID 16). If a peer is choked, it must explicitly answer every queued request with a Reject Request. This allows the requesting client to immediately free its internal queue and reroute the request to an unchoked neighbor, significantly decreasing data latency.

Data Exchange Architecture: Requests, Pieces, and Blocks

The primary objective of the entire architecture is the fault-tolerant transfer of payload data. To accomplish this, the protocol enforces a dual-layered abstraction between "Pieces" and "Blocks".

A "Piece" is the macro-unit defined in the .torrent metainfo file, validated by a corresponding 20-byte SHA-1 hash. Pieces generally range from 256 KiB to 4 MiB. However, attempting to transmit a 4 MiB buffer over a single TCP frame is impossible, and waiting for it to arrive before acknowledging it violates the mechanics of TCP sliding windows. Therefore, the Peer Wire Protocol subdivides Pieces into micro-units known as "Blocks".

The de facto standard dictates that Block sizes must be strictly fixed at 16 KiB (16,384 bytes). This size flawlessly aligns with typical Ethernet MTUs and TCP windowing mechanics. If a rogue client requests a block larger than 128 KiB, standard peers will interpret the action as a Denial-of-Service attack and immediately sever the connection.

The Request Message Memory Alignment

When a client receives an Unchoke message and determines its interest via the bitfield, it initiates the transfer by dispatching a Request message (ID 6). The Request is a fixed 17-byte message structured as follows:

Offset	Size (Bytes)	Data Type	Field Name	Architectural Function

0	4	uint32_t	length_prefix	Fixed at 0x0000000D (Decimal 13).
4	1	uint8_t	message_id	6 (Request).
5	4	uint32_t	index	The zero-based piece index being targeted.
9	4	uint32_t	begin	The byte offset *within* the piece designating the start of the block. This will be 0 for the first block, 16384 for the second, 32768 for the third, etc..
13	4	uint32_t	length	The byte size of the block being requested, universally 16384, barring the final, truncated block of the file.

The Piece Message and Payload Extraction

The remote peer processes the Request, seeks the corresponding byte offset in its local disk array, and streams the data back wrapped in a Piece message (ID 7). The Piece message encapsulates the raw data with a minimal 13-byte header overhead.

Offset	Size (Bytes)	Data Type	Field Name	Architectural Function
0	4	uint32_t	length_prefix	0x00000009 + block_length (Usually 16393 bytes).
4	1	uint8_t	message_id	7 (Piece).
5	4	uint32_t	index	The zero-based piece index.
9	4	uint32_t	begin	The byte offset within the piece.
13	Variable	uint8_t	block	The unadulterated binary payload.

Parsing this structure requires zero-copy or minimal-copy memory techniques. The client reads the index and begin parameters to compute a pointer directly into a pre-allocated heap buffer. The socket stream then writes the block directly into the designated offset. Upon receiving all 16 KiB blocks associated with a specific piece, the client computes the aggregate SHA-1 hash. If the computation yields a precise match against the corresponding 20-byte string in the .torrent metainfo, the piece is flushed to the physical hard drive, and a Have message is broadcasted across the swarm.

Congestion Control, Pipelining, and the End-Game Algorithm

A fundamental challenge of TCP communications over wide-area networks is the bandwidth-delay product (BDP). If a client issues a single 16 KiB Request and idly waits for the corresponding Piece message before issuing the next Request, the transfer speed will

invariably collapse to a fraction of the available bandwidth, bottlenecked by round-trip latency.

To circumvent TCP slow-start and maintain a fully saturated pipeline, BitTorrent mandates an aggressive queuing strategy. A well-architected client will keep a continuous queue of 5 to 10 Request messages pending on the wire at any given moment. This ensures the remote peer's TCP transmit buffers are perpetually loaded, allowing the underlying OS network stack to scale the congestion window up to maximum capacity.

The End-Game Heuristic

This deep pipelining introduces a structural vulnerability during the final stages of a download. When a file reaches 99% completion, the mathematical probability of a single, slow peer stalling the completion of the final missing piece increases dramatically. To neutralize this bottleneck, the protocol activates the "End-Game" algorithm.

When all remaining blocks have been requested but not yet fulfilled, the client intentionally breaks the rules of efficiency and broadcasts duplicate Request messages for the outstanding blocks to *every* active, unchoked peer. While this induces redundant data on the wire, it guarantees that the fastest peer will deliver the final block immediately.

To mitigate the bandwidth penalty of this redundancy, the client heavily utilizes the Cancel message (ID 8). The Cancel message is structurally identical to the Request message, comprising 17 bytes (13 bytes payload + 4 bytes prefix).

Offset	**Size (Bytes)**	**Data Type**	**Field Name**	**Description**
0	4	uint32_t	length_prefix	0x0000000D (Decimal 13).
4	1	uint8_t	message_id	8 (Cancel).
5	4	uint32_t	index	The zero-based piece index.
9	4	uint32_t	begin	The byte offset within the piece.

13	4	uint32_t	length	The requested block size (typically 16384).

The precise microsecond the client finishes reading a Piece message containing an End-Game block, it dispatches Cancel messages for that identical block to all other peers holding the duplicate request. Upon parsing the Cancel message, the remote peers scrub their internal queues, halting the transmission and redirecting their upload capacity to productive transfers.

Conclusion

The architecture of the BitTorrent protocol represents a masterclass in separating network control systems from high-throughput data transmission. By offloading swarm discovery to the highly compressed, binary UDP Tracker Protocol (BEP 15), the system practically eliminates the latency and bandwidth overhead inherent in establishing HTTP connections, compressing entire peer tables into dense 6-byte and 18-byte strides. The meticulous utilization of 64-bit integers and cryptographically randomized transaction IDs ensures long-term statistical viability and robust defense against IP spoofing.

The transition to the data plane via the TCP Peer Wire Protocol showcases a heavily optimized, asynchronous state machine. The un-prefixed 68-byte Handshake establishes identity while utilizing its 8-byte reserved matrix to dynamically negotiate modern extensions, including BEP 5 Distributed Hash Tables, BEP 6 Fast Extensions, and the BEP 10 multiplexed Extension Protocol. Once established, the continuous Length-Value-Payload framing mechanism enables deep, non-blocking pipelining. By abstracting mathematically verified "Pieces" into easily transmittable 16 KiB "Blocks," and orchestrating data flow via granular Choke algorithms and End-Game cancellation heuristics, the protocol maintains saturated TCP congestion windows. Every byte offset, integer conversion, and memory layout detailed in this report forms the cohesive, resilient framework that enables BitTorrent to dominate large-scale decentralized distribution.

Brief Client Implementation Reference: Workflow and Complete Byte Structures

This section provides a condensed, standalone workflow and structural reference for building a BitTorrent client.

1. Client Execution Workflow

1. **Metainfo Parsing:** Decode the .torrent bencoded file to extract the announce URL, piece length, info_hash (20-byte SHA-1 of the info dictionary), and total file length.[1]

2. **Tracker Discovery (UDP):**
   - Dispatch a CONNECT request to the tracker. Receive a connection_id.[1]
   - Dispatch an ANNOUNCE request containing the info_hash and peer_id.[1]
   - Parse the ANNOUNCE response to extract the compact list of peer IPs and ports.[1]
3. **Peer Connection (TCP):**
   - Establish a TCP socket to the extracted peer IP and port.[1]
   - Transmit the 68-byte BitTorrent Handshake.[1]
   - Validate the receiving peer's Handshake and info_hash.[1]
4. **State Synchronization:**
   - Exchange Bitfield messages to map available pieces.[1]
   - Send an Interested message if the peer holds required pieces.[1]
   - Wait to receive an Unchoke message from the peer.[1]
5. **Data Exchange (Piece Downloading):**
   - Request pieces in 16 KiB block increments.[1]
   - Accumulate Piece blocks in memory.[1]
   - Verify the fully assembled piece against the SHA-1 hash.[1]
   - Write the data to disk and broadcast a Have message to all connected peers.[1]

2. Complete Peer Wire Protocol Message Byte Structures

Every message (except the Handshake) follows a strict Length-Value-Payload (LVP) format: <length prefix (4 bytes)><message ID (1 byte)><payload (variable)>.[2] All integers must be evaluated in big-endian network byte order.

Message Type	ID	Total Size	Payload Data Types and Memory Offsets
Keep-Alive	N/A	4 Bytes	length_prefix: 00000000 (4 bytes). Contains no message_id or payload.[2]
Choke	0	5 Bytes	length_prefix: 00000001 (4 bytes). message_id: 0 (1 byte). Zero payload.[2]

Unchoke	1	5 Bytes	length_prefix: 00000001 (4 bytes). message_id: 1 (1 byte). Zero payload.[2]
Interested	2	5 Bytes	length_prefix: 00000001 (4 bytes). message_id: 2 (1 byte). Zero payload.[2]
Not Interested	3	5 Bytes	length_prefix: 00000001 (4 bytes). message_id: 3 (1 byte). Zero payload.[2]
Have	4	9 Bytes	length_prefix: 00000005 (4 bytes). message_id: 4 (1 byte). **Offset 5:** piece_index (4 bytes, uint32_t).[2]
Bitfield	5	5 + X Bytes	length_prefix: 00000001 + X (4 bytes). message_id: 5 (1 byte). **Offset 5:** bitfield (X bytes, uint8_t array where 1 bit = 1 piece).[2]
Request	6	17 Bytes	length_prefix: 0000000D (4 bytes). message_id: 6 (1 byte).

			Offset 5: index (4 bytes, uint32_t). **Offset 9:** begin (4 bytes, uint32_t). **Offset 13:** length (4 bytes, uint32_t).[2]
Piece	7	13 + X Bytes	length_prefix: 00000009 + X (4 bytes). message_id: 7 (1 byte). **Offset 5:** index (4 bytes, uint32_t). **Offset 9:** begin (4 bytes, uint32_t). **Offset 13:** block (X bytes, raw binary payload).[2]
Cancel	8	17 Bytes	Identical memory layout to **Request** but uses message_id: 8.[2]
Port (DHT)	9	7 Bytes	length_prefix: 00000003 (4 bytes). message_id: 9 (1 byte). **Offset 5:** listen-port (2 bytes, uint16_t).[2]
Have All (Fast)	14 / 0x0E	5 Bytes	length_prefix: 00000001 (4 bytes). message_id: 14 (1 byte).

			Negotiated via BEP 6.[3]
Have None (Fast)	15 / 0x0F	5 Bytes	length_prefix: 00000001 (4 bytes). message_id: 15 (1 byte). Negotiated via BEP 6.[3]
Reject Request	16 / 0x10	17 Bytes	Identical memory layout to **Request** but uses message_id: 16. Negotiated via BEP 6.[3]
Extended	20 / 0x14	6 + X Bytes	length_prefix: 00000002 + X (4 bytes). message_id: 20 (1 byte). **Offset 5:** extended_message_id (1 byte, uint8_t). **Offset 6:** payload (X bytes, Bencoded extension dict).[4]

Works cited

1. bittorrent_protocol_flow_report.pdf
2. BitTorrentSpecification - TheoryOrg, accessed on March 16, 2026, https://wiki.theory.org/BitTorrentSpecification
3. bep_0006.rst_post - BitTorrent.org, accessed on March 16, 2026, https://www.bittorrent.org/beps/bep_0006.html
4. bep_0010.rst_post - BitTorrent.org, accessed on March 16, 2026, https://www.bittorrent.org/beps/bep_0010.html